NoSQL

Limitations of RDBMS

- Requires predefined schema

- Works only with structured data

- Difficult to handle Big Data

To deal with unstructured data and large-scale data, NoSQL databases are used.

NoSQL Categories

->Key-Value Databases

  Examples: Redis, Riak

->Columnar Databases

  Examples: HBase, Cassandra

->Document Databases

Examples: MongoDB

->Graph Databases

Examples: Neo4j

Relational vs Document Database

Relational Database

- Tables
- Fixed Schema
- Foreign Keys and Joins
- Vertical Scaling

Example: MySQL, SQL Server

Document Database

- Collections
- Flexible Schema
- Embedded Documents
- Horizontal Scaling

Example: MongoDB

Cluster

A cluster is a group of machines working together to store and process data.

Data Storage

MongoDB stores BSON (Binary JSON) documents

RDBMS stores structured tabular data

MongoDB Structure

MongoDB Hierarchy

Server → Database → Collection → Document (JSON)

Example Document

{

"id":1,

"name":"Anil",

"department":"IT",

"salary":70000

}

Basic MongoDB Commands

Show Databases

show dbs

Show Collections

show collections

Find Data

db.users.find({ name: "Alex" })

Creating Collection

db.createCollection("employees")

Insert Documents

Insert One

```
db.employees.insertOne({
name:'mani',
department:'SDE',
salary:500000,
experience:5
})
```

Insert Many

```
db.employees.insertMany([
{name:'Anvesh',department:'Data',salary:400000},
{name:'Chandra',department:'HR',salary:500000},
{name:'Gokul',department:'SDE',salary:400000}
])
```

Query Operations

Find Employees with Salary Greater Than 70000

```
db.employees.find({salary:{$gt:70000}})
```

Delete Operations

Delete Many

```
db.employees.deleteMany({department:"Finance"
})
```

Delete One

```
db.employees.deleteOne({name:"Rahul"})
```

Update Operations

Increase Salary

```
db.employees.updateMany(
{department:"IT"},
{$inc:{salary:50000}}
)
```

Set Salary

```
db.employees.updateOne(
{name:"Priya"},
{$set:{salary:650000}}
)
```

Aggregation Pipeline

Average Salary by Department

```
db.employees.aggregate([
{
$group:{
_id:"$department",
avgSalary:{$avg:"$salary"}
}
}
])
```

Total Employees by Department

```
db.employees.aggregate([
{
$group:{
_id:"$department",
totalEmployees:{$sum:1}
}
}
])
```

Sort Employees by Salary

```
db.employees.aggregate([
{
$sort:{salary:-1}
}
])
```

Limit Records

db.employees.find().limit(2)

Indexing

Create Index

db.employees.createIndex({department:1})

Compound Index

db.employees.createIndex({department:1, salary:-1})

View Indexes

db.employees.getIndexes()

Count Documents

db.employee.countDocuments()

Mini Project – Employee Collection

Create an Employee collection, insert 50 records, and perform salary analysis.

Salary Analysis

Highest Paid Employee

db.employee.find().sort({salary:-1}).limit(1)

Example Output

_id: ObjectId('69a7b3d97130ed7e32a0db07')

name: 'Max'

department: 'Data'

salary: 8000

Lowest Paid Employee

db.employee.find().sort({salary:1}).limit(1)

Example Output

_id: ObjectId('69a7b3007130ed7e32a0daff')

name: 'Ritul'

department: 'Data'

salary: 2000

Average Salary of All Employees

```
db.employee.aggregate([
{
$group:{
_id:null,
averageSalary:{$avg:"$salary"}
}
}
])
```

Example Output

averageSalary: 4846.153846153846

Average Salary by Department

```
db.employee.aggregate([
{
$group:{
_id:"$department",
avgDeptSalary:{$avg:"$salary"}
}
},
{
$sort:{avgDeptSalary:-1}
}
])
```

Department Headcount

```
db.employee.aggregate([
{
$group:{
```

```
_id:"$department",
HeadCount:{$sum:1}
}
},
{
$sort:{HeadCount:-1}
}
])
```

Top 5 Highest Paid Employees

```
db.employee.find(
{},
{name:1,salary:1,_id:0}
).sort({salary:-1}).limit(5)
```

Create Index on Salary

```
db.employee.createIndex({salary:1})
```